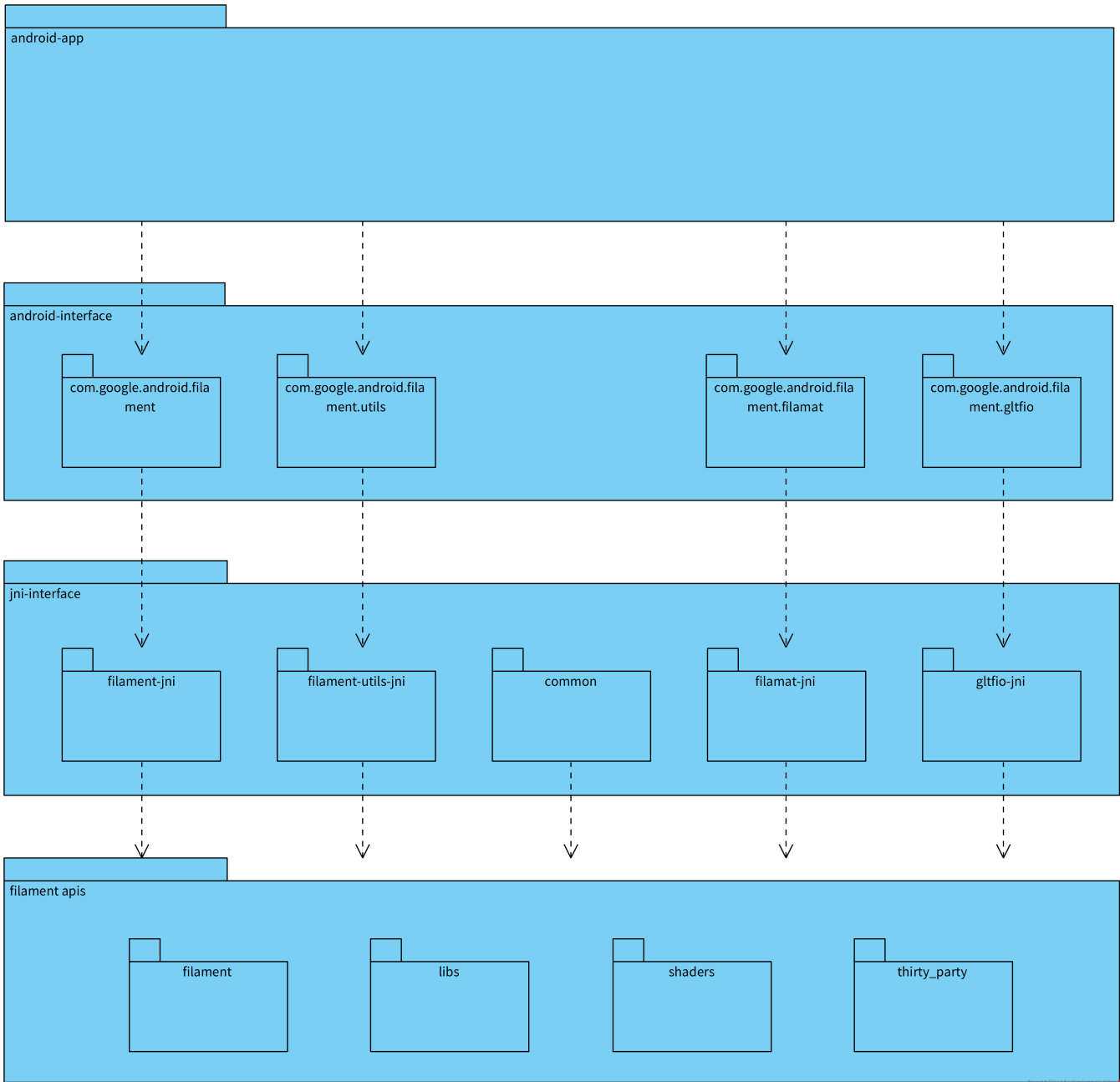


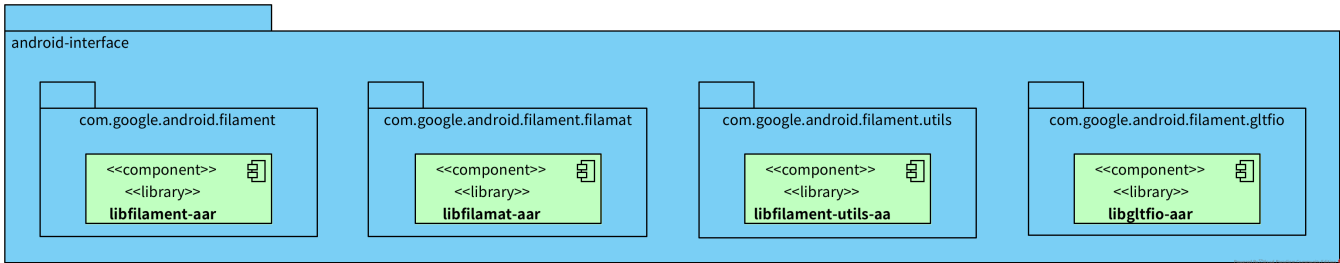
Filament内部模块划分

前面总结了Android Filament上下层结构，回顾下基本组成：



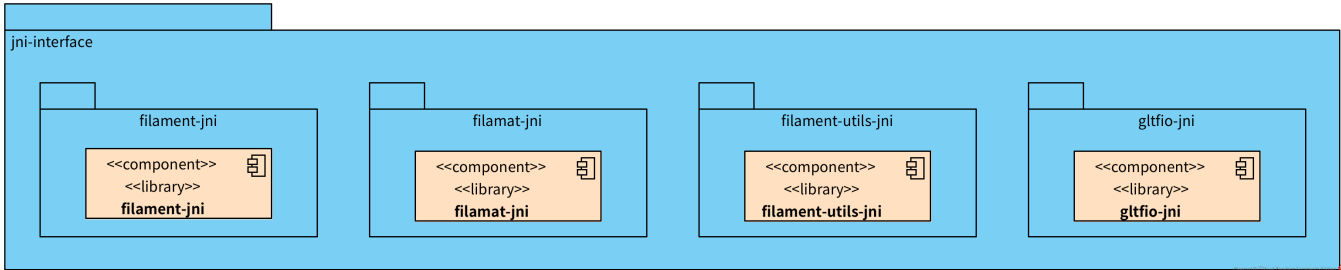
下面来详细分析Filament最底层部分代码的模块划分。

android-interface



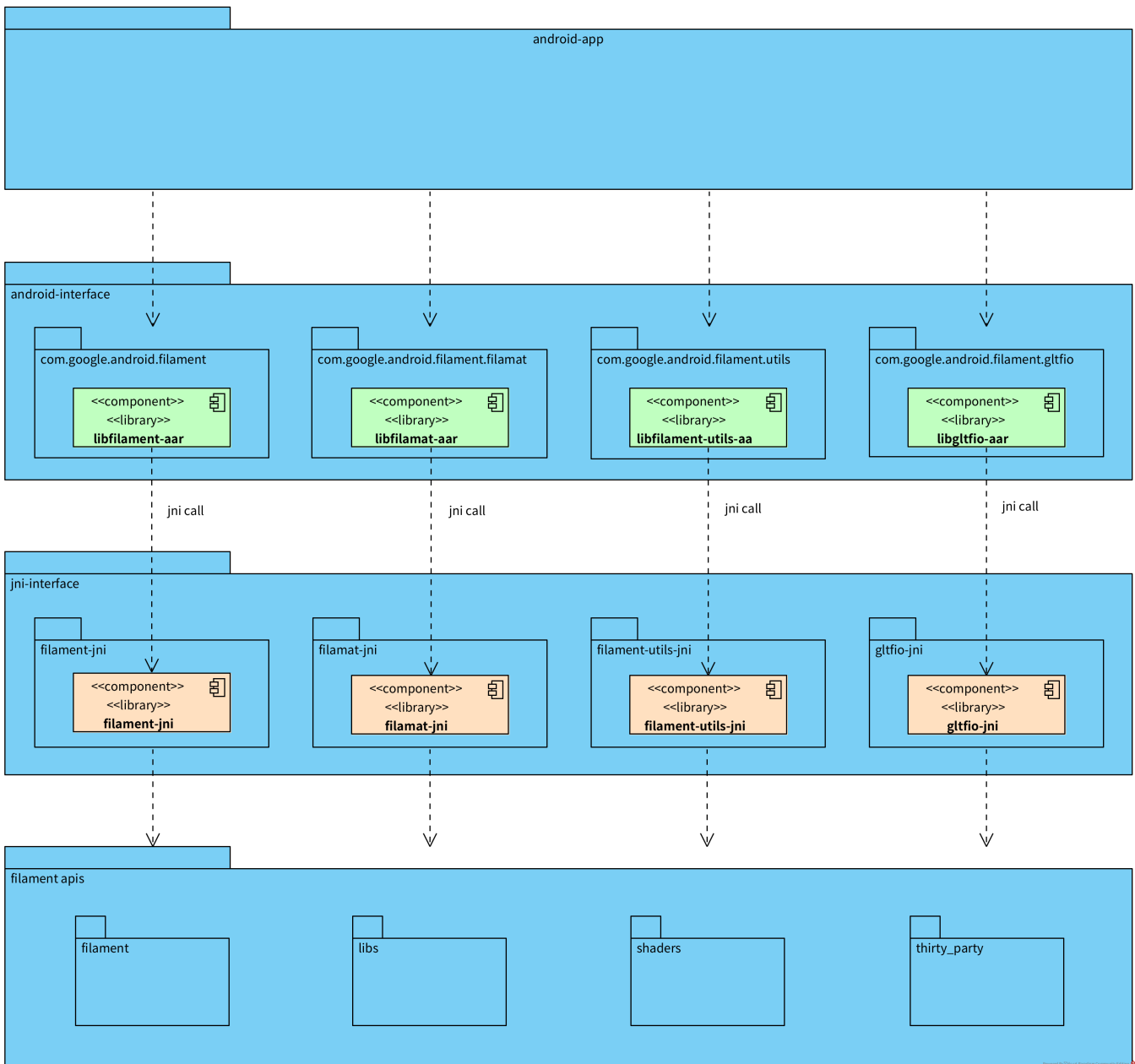
这部分代码分为了四个包，每个包最终都会组成一个android库（aar）。实际上，上层app对android interface的依赖也正是因为app调用的其实就是这几个aar内的类库，在编译过程，就可以将这部分aar和app部分独立开。

jni-interface



这部分代码，在原始包图中，有五个代码包，但最终common会被其它包引用，并不会单独生成库。而剩余四个包最终各自都会生成一个jni调用的动态库（.so文件）。在编译过程，这几个动态库，和上层也没有直接关系，是独立的工程，这样上下层代码就是解耦的，只要将最终的动态库放入上层aar的工程中即可，代码也可以不放入同一个工程。

android interface和jni interface完整的关系



android接口层的aar，通过jni调用，直接调用jni接口层中的动态库，从而实现从java（Kotlin）到c++的调用。

jni接口直接调用filament apis层中的各个静态库，实现实际功能。

filament apis部分，后面继续分析各个目录中静态库分布以及jni层和静态库直接的关系。

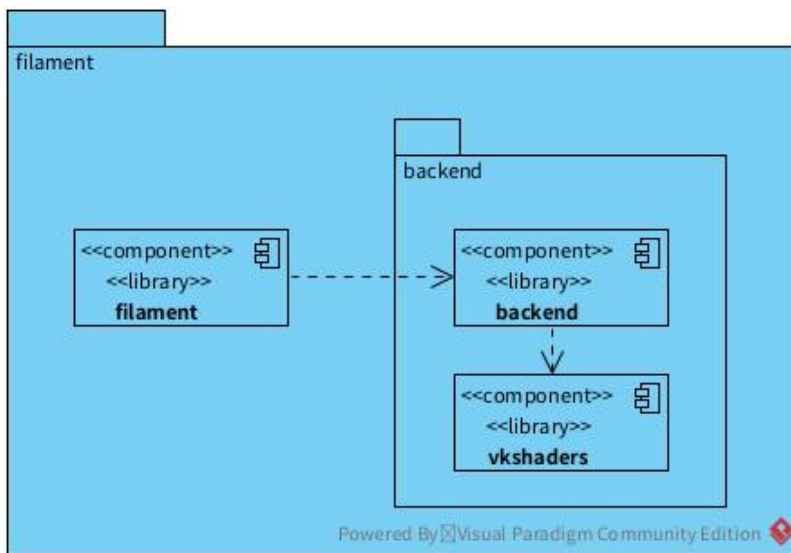
总的说来上面几个层次的代码互相独立，分散在不同工程中，各个模块独立解耦，可以实现单独更新。

filament apis

这部分由于内容比较多，不方便放在一张图上，后面以各个动态库的依赖关系来详细分析，但这里我们需要知道的就是，这部分是由几十个静态库组合而成，几十个静态库分散在四个目录当中，jni层直接链接本层各个库，最终完成功能。这里的静态库和上层动态库也可以放入不同的工程中。

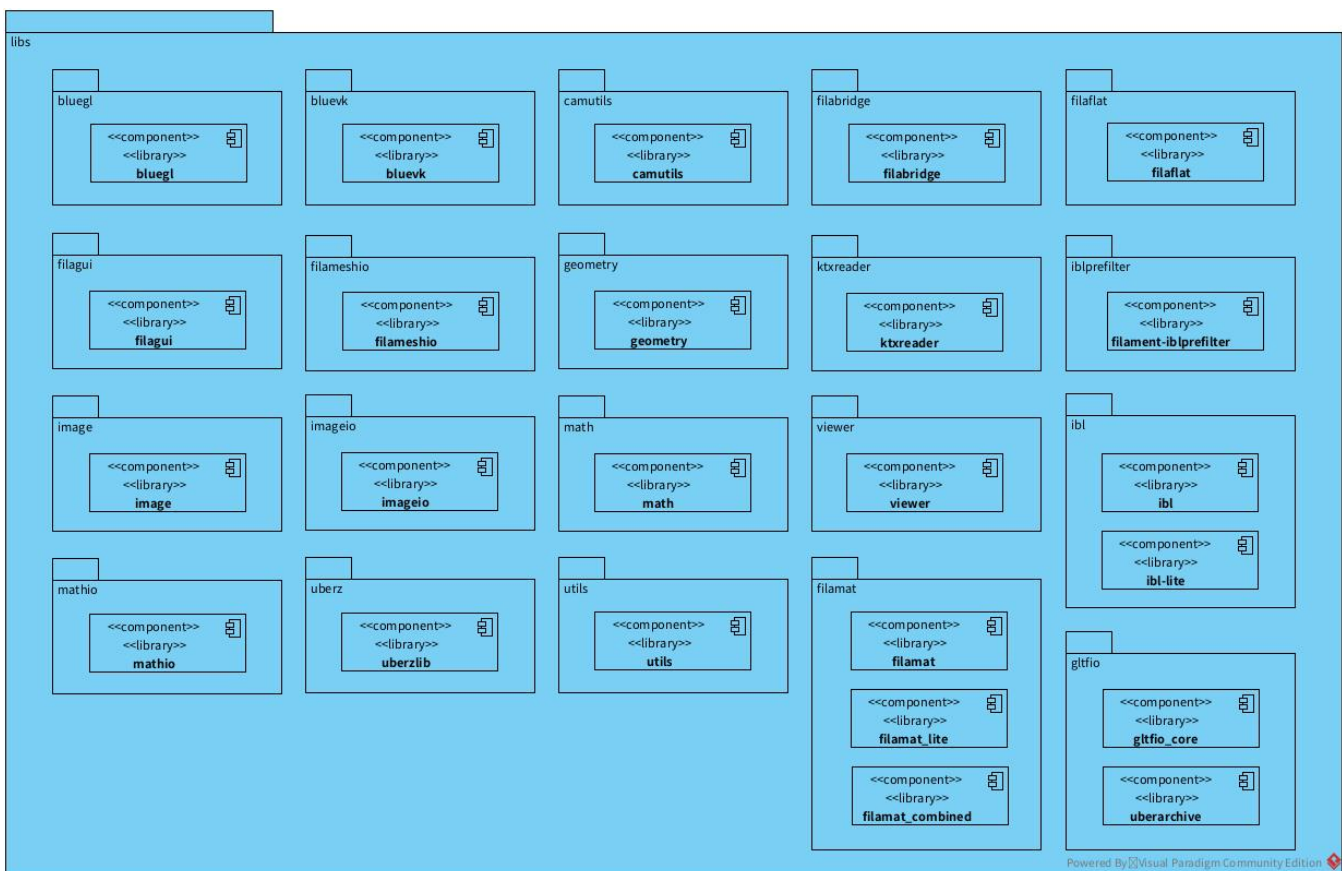
首先，看各个目录中的静态库划分：

filament



filament中，主要包括了一个filament静态库和backend相关的库，backend中又分为了两个静态库，这三个静态库就是完整的filament组成。

libs



libs中，大部分都是一份代码都只会生成一个静态库，而这里也有几个特殊的：

ibl

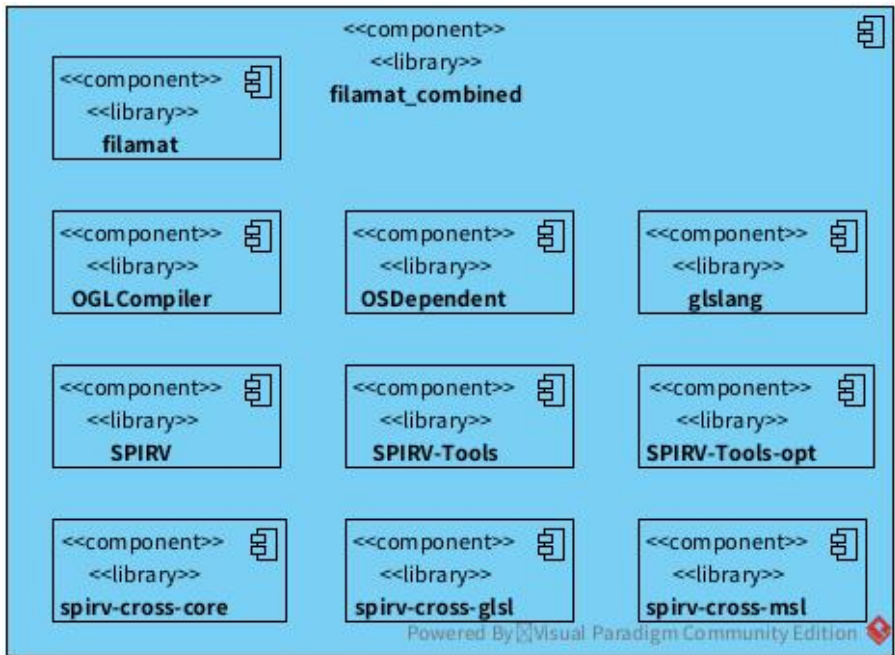
被拆分成了ibl和ibl-lite两个静态库。

gltfio

被拆分成了gltfio_core和uberarchive两个静态库。

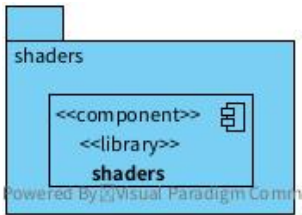
filamat

filamat部分被拆分成了filamat、filamat_lite和filamat_combined三个静态库。而这里filamat_combined又是一个多个静态库组合而成的静态库：



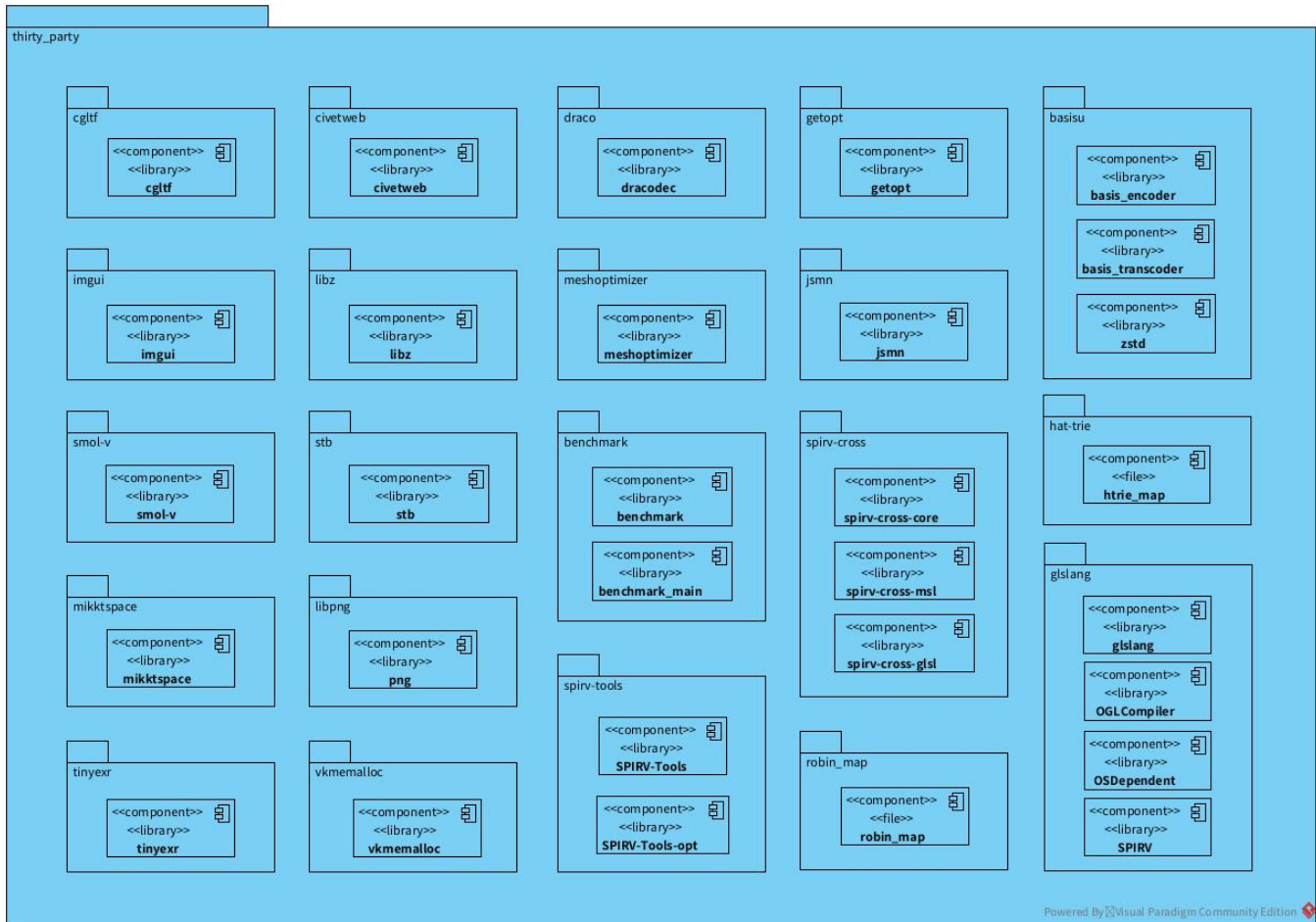
这里可以看到，其中也包含了一些thirty_party部分的静态库。

shaders



shaders中，只有一个静态库。

thirty_party



第三方代码，也是大部分都是代码和静态库一对一，但也有几个，内部会生成多个静态库。

benchmark

拆分成benchmark和benchmark_main两个静态库。

spirv-tools

拆分成SPIRV-Tools、SPIRV-Tools-opt两个静态库。实际上，内部还有SPIRV-Tools-diff、SPIRV-Tools-lint、SPIRV-Tools-reduce和SPIRV-Tools-link几个静态库，我们并没有用到。

spirv-cross

分成了spirv-cross-core、spirv-cross-msl和spirv-cross-gsl三个静态库。

basisu

分为了basis_encoder、basis_transcoder和zstd三个静态库。

gleslang

拆分为gleslang、OGLCompiler、OSDependent和SPIRV四个静态库。

hat-trie

实际上，这个头文件是作为头文件供其它代码直接引用的，并没有实际生成静态库或动态库。

robin_map

同样，也是一组供其它代码直接引用的头文件，也没有生成实际动态库或静态库。